DP 스터디

2026년 4월 3일

증가하는 부분 수열의 개수(22971)

문제

길이가 $N(1 \le N \le 5\,000)$인 수열 A가 주어진다. 수열의 i번째 원소 $A_i(1 \le A_i \le 5\,000)$로 끝나는 증가하는 부분 수열의 개수를 출력하는 프로그램을 작성하자.
단, 수가 너무 커질 수 있으니 $998\,244\,353$으로 나눈 나머지를 출력한다.

부분 수열이란 주어진 수열에서 1개 이상의 원소를 골라 원래 순서대로 나열하여 얻은 수열을 말한다.

증가하는 부분 수열이란 맨 처음 원소를 제외한 모든 원소가 바로 전 원소보다 큰 수열을 말한다. 다시 말해 길이가 N인 부분 수열 A가 있을 때, $A_{i-1} < A_i$ $(2 \le i \le N)$ 를 만족하면 A는 증가하는 부분 수열이다.

길이가 1인 부분 수열은 항상 증가하는 부분 수열임에 주의하자.

예제 입력/출력 1

입력

```
5
1 2 3 4 5
```

출력

```
1 2 4 8 16
```

예제 입력/출력 3

입력

```
5
1 2 2 4 3
```

출력

```
1 2 2 6 6
```

김시온 소스 코드 1

```java
import java.util.*;
import java.io.*;

public class Main {
    static final int mod = 998244353;

    static int n;
    static int[] a, c;

    // dp(i) = a_i에서 끝나는 LIS의 개수
    static int dp(int i) {
        if (c[i] > 0) {
            return c[i];
        }
        int sum = 1;
        for (int j = 0; j < i; j++) {
            if (a[j] < a[i]) {
                sum = (sum + dp(j)) % mod;
            }
        }
        return c[i] = sum;
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringBuilder bw = new StringBuilder();
        n = Integer.parseInt(br.readLine());
        a = new int[n];
        StringTokenizer st = new StringTokenizer(br.readLine());
```

김시온 소스 코드 2

```
        for (int i = 0; i < n; i++) {
            a[i] = Integer.parseInt(st.nextToken());
        }
        c = new int[n];
        for (int i = 0; i < n; i++) {
            bw.append(dp(i));
            if (i == n - 1) {
                bw.append('\n');
            } else {
                bw.append(' ');
            }
        }
        System.out.print(bw);
    }

}
```

원찬혁 소스 코드 1

```java
package algo_test;

import java.io.*;
import java.util.*;

public class Main {
    static int ary[], dp[];
    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringBuilder sb = new StringBuilder();
        int n = Integer.parseInt(br.readLine());
        StringTokenizer st = new StringTokenizer(br.readLine());
        ary = new int[n];
        dp = new int[n];
        for(int i = 0; i < n; i++)
            ary[i] = Integer.parseInt(st.nextToken());
        dp[0] = 1;
        sb.append(1).append(" ");
        for(int i = 1; i < n; i++) {
            dp[i] = 1;
            for(int j = 0; j < i; j++) {
                if(ary[j] < ary[i])
                    dp[i] += dp[j];
                dp[i] %= 998244353;
            }
            sb.append(dp[i]).append(" ");
        }
        System.out.println(sb);
```

원찬혁 소스 코드 2

```
        }
    }
```

손현준 소스 코드 1

```java
package baekjoon;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.StringTokenizer;

public class P22971 {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringBuilder sb = new StringBuilder();
        StringTokenizer st;

        final int MOD = 998244353;

        int N = Integer.parseInt(br.readLine());
        int[] num = new int[N];
        long[] dp = new long[N];

        st = new StringTokenizer(br.readLine(), " ");
        for (int n = 0; n < N; n++) {
            num[n] = Integer.parseInt(st.nextToken());
        }

        for (int n = 0; n < N; n++) {
            int tNum = num[n];
            long sum = 1;

            for (int i = n - 1; i >= 0; i--) {
```

손현준 소스 코드 2

```
                if (num[i] < tNum) sum = (sum + dp[i]) % MOD;
            }

            dp[n] = sum;
            sb.append(sum).append(" ");
        }
        System.out.println(sb.toString());
    }
}
```

김지훈 소스 코드 1

```java
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.util.Arrays;
import java.util.StringTokenizer;

public class BOJ_22971 {
    static int N;
    static int[] A;
    static long[] memo;
    static final int MOD = 998244353;

    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        N = Integer.parseInt(br.readLine());
        A = new int[N];
        memo = new long[N];
        Arrays.fill(memo, -1);

        StringTokenizer st = new StringTokenizer(br.readLine());
        for (int i = 0; i < N; i++) {
            A[i] = Integer.parseInt(st.nextToken());
        }

        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < N; i++) {
            sb.append(solve(i)).append(" ");
        }
        System.out.println(sb);
```

김지훈 소스 코드 2

```
30      }
31
32      private static long solve(int i) {
33
34          if (memo[i] != -1)
35              return memo[i];
36
37          long count = 1;
38          for (int j = 0; j < i; j++) {
39              if (A[j] < A[i]) {
40                  count = (count + solve(j)) % MOD;
41              }
42          }
43
44          return memo[i] = count;
45      }
46  }
```

김준형 소스 코드 1

```java
import java.io.*;
import java.util.*;

public class Main {
    static int n;
    static long[] dp;
    static int[] arr;
    static final int DIV = 998244353;
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringBuilder sb = new StringBuilder();
        n = Integer.parseInt(br.readLine());
        dp = new long[n+1];
        Arrays.fill(dp,1);
        arr = new int[n];
        StringTokenizer st = new StringTokenizer(br.readLine());

        for(int i=0; i<n; i++) {
            arr[i] = Integer.parseInt(st.nextToken());
        }

        for(int i=0; i<n; i++) {
            int temp = arr[i];
            for(int j=0; j<i; j++) {
                if(arr[j]<temp) {
                    dp[i]=(dp[i]+dp[j])%DIV;
                }
            }
            sb.append(dp[i]%998244353).append(" ");
```

김준형 소스 코드 2

```
        }

        System.out.print(sb);
    }

}
```

서민종 소스 코드 1

```java
package BOJ22971;

import java.util.*;
import java.io.*;

public class Main {
    static long INF = 998244353;
    static int N;
    static int[] numbers;
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(System.out));
        N = Integer.parseInt(br.readLine());
        numbers = new int[N];
        StringTokenizer st = new StringTokenizer(br.readLine());
        for(int i=0; i<N; i++) {
            numbers[i] = Integer.parseInt(st.nextToken());
        }
        long[] dp = new long[N];
        dp[0] = 1;
        for(int i=1; i<N; i++) {
            long sum = 0;
            for(int j=0; j<i; j++) {
                if(numbers[j] < numbers[i]) {
                    sum += dp[j];
                }
            }
            dp[i] = (sum + 1) % INF;
```

서민종 소스 코드 2

```
29          }
30          StringBuilder sb = new StringBuilder();
31          for(int i=0; i<N; i++) {
32              sb.append(dp[i]).append(" ");
33          }
34          bw.write(sb.toString());
35          bw.flush();
36      }
37  }
38
```

징검다리 건너기(21317)

문제

심마니 영재는 산삼을 찾아다닌다.
산삼을 찾던 영재는 $N(1 \le N \le 20)$개의 돌이 일렬로 나열되어 있는 강가를 발견했고, 마지막 돌 틈 사이에 산삼이 있다는 사실을 알게 되었다.
마지막 돌 틈 사이에 있는 산삼을 캐기 위해 영재는 돌과 돌 사이를 점프하면서 이동하며, 점프의 종류는 3가지가 있다. 현재 위치에서 다음 돌로 이동하는 작은 점프, 1개의 돌을 건너뛰어 이동하는 큰 점프, 2개의 돌을 건너뛰어 이동하는 매우 큰 점프가 있다.

각 점프를 할 때는 에너지를 소비하는데, 작은 점프와 큰 점프 시 소비되는 에너지는 점프를 하는 돌의 번호마다 다르다. 매우 큰 점프는 단 한 번의 기회가 주어지며, 이때는 점프를 하는 돌의 번호와 상관없이 $K(1 \le K \le 5\,000)$만큼의 에너지를 소비한다. 작은 점프와 큰 점프 시 필요한 에너지도 $5\,000$을 넘지 않는 자연수이다.

에너지를 최대한 아껴야 하는 영재가 첫 번째 돌에서 출발하여 산삼을 얻기 위해 필요한 최소 에너지를 구하여라.

예제 입력/출력 1

입력

```
5
1 2
2 3
4 5
6 7
4
```

출력

```
5
```

김시온 소스 코드 1

```java
import java.util.*;
import java.io.*;

public class Main {
    static int n, k;
    static int[] a, b;
    static int[][] c;

    // dp(i, j) = (i, i + 1)번째 돌 사이를 뛰어 넘으려고 한다. j는 매우 큰 점프를 했는지 여부.
    static int dp(int i, int j) {
        if (i == n - 1) {
            return 0;
        }
        if (c[j][i] != -1) {
            return c[j][i];
        }
        // 1. 작은 점프
        int min = a[i] + dp(i + 1, j);
        // 2. 큰 점프
        if (i + 2 < n) {
            min = Math.min(min, b[i] + dp(i + 2, j));
        }
        // 3. 매우 큰 점프
        if (i + 3 < n && j == 0) {
            min = Math.min(min, k + dp(i + 3, 1));
        }
        return c[j][i] = min;
    }

```

김시온 소스 코드 2

```
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        n = Integer.parseInt(br.readLine());
        a = new int[n];
        b = new int[n];
        c = new int[2][n];
        Arrays.fill(c[0], -1);
        Arrays.fill(c[1], -1);

        for (int i = 0; i < n - 1; i++) {
            StringTokenizer st = new StringTokenizer(br.readLine());
            a[i] = Integer.parseInt(st.nextToken());
            b[i] = Integer.parseInt(st.nextToken());
        }
        k = Integer.parseInt(br.readLine());
        System.out.println(dp(0, 0));
    }
}
```

원찬혁 소스 코드 1

```java
package algo_test;

import java.io.*;
import java.util.*;

public class Main {
    static int miniJump[], bigJump[], dp[][], cheatJump;
    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st;
        int n = Integer.parseInt(br.readLine());
        miniJump = new int[n];
        bigJump = new int[n];
        dp = new int[n][2];
        for(int i = 0; i < n - 1; i++) {
            st = new StringTokenizer(br.readLine());
            miniJump[i] = Integer.parseInt(st.nextToken());
            bigJump[i] = Integer.parseInt(st.nextToken());
        }
        cheatJump = Integer.parseInt(br.readLine());
        System.out.println(Math.min(sol(n - 1, 1), sol(n - 1, 0)));
    }

    static int sol(int idx, int m) {
        if(idx <= 0)
            return 0;
        if(dp[idx][m] > 0)
            return dp[idx][m];
        dp[idx][m] = 1000000000;
```

원찬혁 소스 코드 2

```
        if(m > 0) {
            if(idx >= 3)
                dp[idx][m] = sol(idx - 3, 0) + cheatJump;
        }

        dp[idx][m] = Math.min(dp[idx][m], sol(idx - 1, m) + miniJump[idx - 1]);
        if(idx > 1)
            dp[idx][m] = Math.min(dp[idx][m], sol(idx - 2, m) + bigJump[idx - 2]);

        return dp[idx][m];
    }
}
```

손현준 소스 코드 1

```java
package baekjoon;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.StringTokenizer;

public class P21317 {
    static class JumpCost {
        int jumpS, jumpL;

        public JumpCost(int jumpS, int jumpL) {
            this.jumpS = jumpS;
            this.jumpL = jumpL;
        }
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st;

        int N = Integer.parseInt(br.readLine());
        JumpCost[] jumpCost = new JumpCost[N - 1];
        for (int n = 0; n < (N - 1); n++) {
            st = new StringTokenizer(br.readLine(), " ");
            int jumpS = Integer.parseInt(st.nextToken());
            int jumpL = Integer.parseInt(st.nextToken());
            jumpCost[n] = new JumpCost(jumpS, jumpL);
        }
```

손현준 소스 코드 2

```java
        int K = Integer.parseInt(br.readLine());

        // dp[바위 번호][매우 큰 점프 사용 여부]
        int[][] dp = new int[N][2];

        for (int n = 0; n < (N - 1); n++) {
            int nCost = 0;

            // 작은 점프 - K 사용 안한 상태
            nCost = dp[n][0] + jumpCost[n].jumpS;
            dp[n + 1][0] = (dp[n + 1][0] == 0) ? nCost : Integer.min(dp[n + 1][0], nCost);
            // 작은 점프 - K 사용한 상태
            if (dp[n][1] != 0) {
                nCost = dp[n][1] + jumpCost[n].jumpS;
                dp[n + 1][1] = (dp[n + 1][1] == 0) ? nCost : Integer.min(dp[n + 1][1], nCost);
            }

            // 큰 점프 - K 사용 안한 상태
            if (n == (N - 2)) continue;
            nCost = dp[n][0] + jumpCost[n].jumpL;
            dp[n + 2][0] = (dp[n + 2][0] == 0) ? nCost : Integer.min(dp[n + 2][0], nCost);
            // 큰 점프 - K 사용한 상태
            if (dp[n][1] != 0) {
                nCost = dp[n][1] + jumpCost[n].jumpL;
                dp[n + 2][1] = (dp[n + 2][1] == 0) ? nCost : Integer.min(dp[n + 2][1], nCost);
```

손현준 소스 코드 3

```java
            }

            // 매우 큰 점프
            if (n == (N - 3)) continue;
            nCost = dp[n][0] + K;
            dp[n + 3][1] = (dp[n + 3][1] == 0) ? nCost : Integer.min(dp[n + 3][1], nCost);
        }

        System.out.println((dp[N - 1][1] == 0) ? dp[N - 1][0] : Integer.min(dp[N - 1][0], dp[N - 1][1]));
    }
}
```

김준형 소스 코드 1

```java
import java.io.*;
import java.util.*;

public class Main {
    static class Jump{
        int s;
        int m;
        public Jump(int s, int m) {
            this.s = s;
            this.m = m;
        }
    }
    static int n,k;
    static Jump[] jumpArr;
    static int dp[][];
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st;
        n = Integer.parseInt(br.readLine());
        dp = new int[n+1][2];
        for(int i=0; i<n+1; i++) {
            Arrays.fill(dp[i], 100000);
        }
        jumpArr = new Jump[n-1];

        for(int i=0; i<n-1; i++) {
            st = new StringTokenizer(br.readLine());
            jumpArr[i] = new
            ↪  Jump(Integer.parseInt(st.nextToken()),Integer.parseInt(st.nextToken()));
```

김준형 소스 코드 2

```
        }
        k = Integer.parseInt(br.readLine());

        dp[1][0] = 0; dp[1][1] = 0;
        for(int i=1; i<=n; i++) {
            // 슈퍼 점프 사용 안했을 때
            if(i+1<=n) {
                dp[i+1][0] = Math.min(dp[i+1][0],dp[i][0]+jumpArr[i-1].s);
            }
            if(i+2<=n) {
                dp[i+2][0] = Math.min(dp[i+2][0],dp[i][0]+jumpArr[i-1].m);
            }
            if(i+3<=n) {
                dp[i+3][1] = Math.min(dp[i+3][1],dp[i][0]+k);
            }

            // 슈퍼 점프 사용 했을 때
            if(i+1<=n && i+1>3) {
                dp[i+1][1] = Math.min(dp[i+1][1],dp[i][1]+jumpArr[i-1].s);
            }
            if(i+2<=n && i+2>3) {
                dp[i+2][1] = Math.min(dp[i+2][1],dp[i][1]+jumpArr[i-1].m);
            }
        }

        System.out.print(Math.min(dp[n][0], dp[n][1]));
    }
```

김준형 소스 코드 3

```

}
```

서민종 소스 코드 1

```java
package BOJ21317;

import java.util.*;
import java.io.*;

public class Main {
    static int N, K;
    static int[][] jump;
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(System.out));
        N = Integer.parseInt(br.readLine());
        jump = new int[N][2];
        for(int i=1; i<=N-1; i++) {
            StringTokenizer st = new StringTokenizer(br.readLine());
            jump[i][0] = Integer.parseInt(st.nextToken());
            jump[i][1] = Integer.parseInt(st.nextToken());
        }
        K = Integer.parseInt(br.readLine());
        int[][] dp = new int[2][N+1];
        Arrays.fill(dp[0], 1000000);
        Arrays.fill(dp[1], 1000000);
        dp[0][1] = 0;
        if(N >= 2) {
            dp[0][2] = jump[1][0];
        }
        if(N >= 3) {
            dp[0][3] = Math.min(dp[0][1] + jump[1][1], dp[0][2] + jump[2][0]);
```

서민종 소스 코드 2

```
        }
        for(int i=3; i<=N; i++) {
            dp[0][i] = Math.min(dp[0][i-2] + jump[i-2][1], dp[0][i-1] + jump[i-1][0]);
            dp[1][i] = Math.min(dp[1][i-2] + jump[i-2][1], dp[1][i-1] + jump[i-1][0]);
            dp[1][i] = Math.min(dp[1][i], dp[0][i-3] + K);
        }
        int answer = Math.min(dp[0][N], dp[1][N]);
        bw.write(answer + "\n");
        bw.flush();
    }
}
```

사수빈탕(14585)

문제

수빈이는 좌표평면 위의 원점 $(0, 0)$에 앉아 있다. 좌표평면에는 $N(0 \le N \le 300)$개의 사탕 바구니가 있고, 각 사탕 바구니에는 처음에 $M(1 \le M \le 1\,000\,000)$개의 사탕이 들어 있다.
각 사탕 바구니는 $(x_1, y_1), (x_2, y_2), \cdots, (x_N, y_N)(0 \le x_i, y_i \le 300)$에 있고, 위치는 서로 중복되지 않으며 $(0, 0)$에는 사탕이 없다.

시간이 1만큼 지날 때마다 사탕이 남아 있는 모든 사탕 바구니에서 사탕은 한 개씩 녹아서 없어진다. 수빈이는 사탕 바구니에 있는 사탕을 도착하자마자 순식간에 모두 먹을 수 있다.

수빈이가 1만큼 움직일 때 시간도 1만큼 지나가며, 수빈이는 위쪽 또는 오른쪽으로만 움직일 수 있다.
수빈이가 먹을 수 있는 사탕의 최대 개수를 구하는 프로그램을 작성하시오.

예제 입력/출력 1

입력	출력
3 15 1 1 3 1 1 6	24

김시온 소스 코드 1

```java
import java.util.*;
import java.io.*;

public class Main {
    static int n, m;
    static boolean[][] a;
    static long[][] c;

    // dp(x, y) = (x, y)에서 시작해서 최대한 먹을 수 있는 사탕의 양
    static long dp(int x, int y) {
        if (c[x][y] != -1) {
            return c[x][y];
        }
        long max = 0;
        if (x + 1 <= 300) {
            max = Math.max(max, dp(x + 1, y));
        }
        if (y + 1 <= 300) {
            max = Math.max(max, dp(x, y + 1));
        }
        // x + 1, y + 1방향으로만 이동 가능하기 때문에 현재 시간 t = x + y
        long candy = a[x][y] ? Math.max(m - (x + y), 0) : 0;
        return c[x][y] = candy + max;
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        n = Integer.parseInt(st.nextToken());
```

김시온 소스 코드 2

```
        m = Integer.parseInt(st.nextToken());

        a = new boolean[301][301];
        for (int i = 0; i < n; i++) {
            st = new StringTokenizer(br.readLine());
            int x = Integer.parseInt(st.nextToken());
            int y = Integer.parseInt(st.nextToken());
            a[x][y] = true;
        }

        // 메모이제이션은 문제의 정답이 될 수 없는 값으로 초기화해야 한다.
        c = new long[301][301];
        for (int x = 0; x <= 300; x++) {
            Arrays.fill(c[x], -1);
        }

        System.out.println(dp(0, 0));
    }
}
```

원찬혁 소스 코드 1

```java
import java.io.*;
import java.util.*;

public class Main {
    static int n, m;
    static int dp[][];
    static int ary[][];
    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
        n = Integer.parseInt(st.nextToken());
        m = Integer.parseInt(st.nextToken());
        dp = new int[301][301];
        ary = new int[301][301];
        for(int i = 0; i <= 300; i++) {
            for(int j = 0; j <= 300; j++)
                dp[i][j] = -1;
        }
        for(int i = 0; i < n; i++) {
            st = new StringTokenizer(br.readLine());
            ary[Integer.parseInt(st.nextToken())]
            ↪  [Integer.parseInt(st.nextToken())] = 1;
        }
        System.out.println(sol(300, 300));
    }
    static int sol(int x, int y) {
        if(x == 0 && y == 0) return 0;
        if(dp[x][y] >= 0) return dp[x][y];
        dp[x][y] = 0;
```

원찬혁 소스 코드 2

```
        if(x > 0)
            dp[x][y] = Math.max(dp[x][y], sol(x - 1, y));
        if(y > 0)
            dp[x][y] = Math.max(dp[x][y], sol(x, y - 1));
        if(ary[x][y] == 1)
            dp[x][y] += Math.max(0, m - (x + y));
        return dp[x][y];
    }
}
```

손현준 소스 코드 (BottomUp) 1

```java
package baekjoon;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.Arrays;
import java.util.HashMap;
import java.util.StringTokenizer;

public class P14585_BottomUp {
    static class Candy {
        long c; // 누적 사탕 개수
        int t; // 걸린 시간

        public Candy(long c, int t) {
            this.c = c;
            this.t = t;
        }
    }

    static class Point {
        int x, y;
        Point(int x, int y) {
            this.x = x;
            this.y = y;
        }
    }

```

손현준 소스 코드 (BottomUp) 2

```java
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st;

        st = new StringTokenizer(br.readLine(), " ");
        int N = Integer.parseInt(st.nextToken());
        int M = Integer.parseInt(st.nextToken());

        Point[] points = new Point[N];
        HashMap<Integer, Candy> baskets = new HashMap<>(); // 해당 정점까지 먹은 사탕 최대값


        long maxCandy = 0;
        for (int n = 0; n < N; n++) {
            st = new StringTokenizer(br.readLine(), " ");
            int xIn = Integer.parseInt(st.nextToken());
            int yIn = Integer.parseInt(st.nextToken());
            points[n] = new Point(xIn, yIn);

            // 좌표 (0, 0) 에서 바로 가는 경우에 대한 계산을 하고 저장
            int time = xIn + yIn;
            long candy = (M - time <= 0) ? 0L : M - time;
            baskets.put(getKey(xIn, yIn), new Candy(candy, time));
            maxCandy = Math.max(maxCandy, candy);
//          System.out.println("(" + xIn + ", " + yIn + ") - " + candy);
        }

```

손현준 소스 코드 (BottomUp) 3

```
        Arrays.sort(points, (o1, o2) -> {
            if (o1.x != o2.x) return Integer.compare(o1.x, o2.x);
            return Integer.compare(o1.y, o2.y);
        });

        for (int i = 0; i < N; i++) {
            int xFrom = points[i].x;
            int yFrom = points[i].y;

            // 존재하지 않는 좌표라면 continue
            int keyFrom = getKey(xFrom, yFrom);
            Candy from = baskets.get(keyFrom);
            if (from == null) continue;

            for (int j = i + 1; j < N; j++) {
                int xTo = points[j].x;
                int yTo = points[j].y;

                // 오른쪽 위에 있는 점만 이동 가능
                 if (xTo < xFrom || yTo < yFrom) continue;

                // 존재하지 않는 좌표라면 continue
                int keyTo = getKey(xTo, yTo);
                Candy next = baskets.get(keyTo);
                if (next == null) continue;

                int nextT = from.t + (xTo - xFrom) + (yTo - yFrom);
                long nextC = M - nextT;
```

손현준 소스 코드 (BottomUp) 4

```
                    if (nextC <= 0) continue; // 사탕이 모두 녹아서 사라졌다면 continue

                    nextC += from.c; // 이전 사탕 개수 더해주기
                    if (next.c < nextC) {
                        baskets.put(keyTo, new Candy(nextC, nextT));
                        maxCandy = Math.max(maxCandy, nextC);
//                      System.out.println("(" + xFrom + ", " + yFrom + ") > (" + xTo + ", " + yTo + ") - " + nextC);
                    }
                }
            }
            System.out.println(maxCandy);
        }

        static int getKey(int x, int y) {
            return (x << 9) | y;
        }
    }
```

손현준 소스 코드 (DFS) 1

```java
package baekjoon;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.StringTokenizer;

public class P14585_DFS {
    static int N, M;
    static Basket[] basket;
    static long dp[];

    static class Basket {
        int x, y;

        Basket(int x, int y) {
            this.x = x;
            this.y = y;
        }
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st;

        st = new StringTokenizer(br.readLine(), " ");
        N = Integer.parseInt(st.nextToken());
        M = Integer.parseInt(st.nextToken());
```

손현준 소스 코드 (DFS) 2

```java

        basket = new Basket[N + 1];
        basket[0] = new Basket(0, 0);
        for (int n = 1; n <= N; n++) {
            st = new StringTokenizer(br.readLine(), " ");
            int x = Integer.parseInt(st.nextToken());
            int y = Integer.parseInt(st.nextToken());
            basket[n] = new Basket(x, y);
        }

        dp = new long[N + 1];
        System.out.println(func(0, 0, 0));
    }

    /*
     * 걸린 시간
     * 마지막 방문 바구니의 인덱스
     * 누적 사탕
     */
    static long func(int t, int i, long c) {
        int nowX = basket[i].x;
        int nowY = basket[i].y;

        if (t >= M) {
            return c;
        }

        // 다음으로 바구니를 방문하는 경우를 모두 확인
```

손현준 소스 코드 (DFS) 3

```
        long result = c;
        for (int n = 1; n <= N; n++) {
            int nextX = basket[n].x;
            int nextY = basket[n].y;

            // 우측 상단으로만 이동
            if (nextX < nowX || nextY < nowY || (nextX == nowX && nextY == nowY))
            ↪  continue;

            // 거리 계산
            int dist = t + (nextX - nowX) + (nextY - nowY);
            long nCandy = c + M - dist;

            if (dp[n] > nCandy) continue;
            dp[n] = nCandy;

            result = Math.max(result, func(dist, n, nCandy));
        }

        return result;
    }
}
```

손현준 소스 코드 (TopDown) 1

```java
package baekjoon;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.StringTokenizer;

public class P14585_TopDown {
    static int N, M;
    static int[][] basket;
    static int[][] dp;

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st;

        st = new StringTokenizer(br.readLine());
        N = Integer.parseInt(st.nextToken());
        M = Integer.parseInt(st.nextToken());

        basket = new int[301][301];
        for (int n = 0; n < N; n++) {
            st = new StringTokenizer(br.readLine(), " ");
            int x = Integer.parseInt(st.nextToken());
            int y = Integer.parseInt(st.nextToken());
            basket[x][y] = 1; // 해당 좌표에 바구니가 있는지 기록
        }

```

손현준 소스 코드 (TopDown) 2

```
        dp = new int[301][301];
        for (int x = 0; x <= 300; x++) {
            for (int y = 0; y <= 300; y++) {
                dp[x][y] = -1;
            }
        }

        System.out.println(func(300, 300));
    }

    static int func(int x, int y) {
        if (x == 0 && y == 0) return 0;
        if (dp[x][y] >= 0) return dp[x][y];

        if (x > 0) dp[x][y] = Math.max(dp[x][y], func(x - 1, y));
        if (y > 0) dp[x][y] = Math.max(dp[x][y], func(x, y - 1));
        if (basket[x][y] == 1) { // 바구니가 존재할때, 바구니에 남은 사탕 수 더해줌
            int t = x + y;
            dp[x][y] += (M < t) ? 0 : M - t;
        }

        return dp[x][y];
    }
}
```

김준형 소스 코드 1

```java
import java.io.*;
import java.util.*;

public class Main {
    static class Cord{
        int y;
        int x;
        public Cord(int y, int x) {
            this.y = y;
            this.x = x;
        }
    }
    static int n,m,ans = -1;
    static Cord[] candies;
    static int cache[][];
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());

        n = Integer.parseInt(st.nextToken());
        m = Integer.parseInt(st.nextToken());
        candies = new Cord[n];
        cache = new int[301][301];
        for(int i=0; i<301; i++) {
            Arrays.fill(cache[i], -1);
        }
        for(int i=0; i<n; i++) {
            st = new StringTokenizer(br.readLine());
            int x = Integer.parseInt(st.nextToken());
```

김준형 소스 코드 2

```
30                int y = Integer.parseInt(st.nextToken());
31                candies[i] = new Cord(y,x);
32            }
33
34            backTracking(0,0,0,0);
35            System.out.print(ans);
36        }
37
38        static void backTracking(int y, int x, int ate, int t) {
39            if(cache[y][x] > ate || m<t) {
40                return;
41            }
42
43            cache[y][x] = ate;
44            ans = Math.max(ans, ate);
45
46            for(int i=0; i<n; i++) {
47                Cord next = candies[i];
48                int ny = next.y; int nx = next.x;
49                if(ny>y && nx>=x || nx>x && ny>=y) {
50                    int time = getDistance(ny, nx, y, x);
51                    backTracking(ny, nx, ate+m-(t+time), t+time);
52                }
53            }
54        }
55
56        static int getDistance(int y2,int x2, int y1, int x1) {
57            return Math.abs(y2-y1)+Math.abs(x2-x1);
```

김준형 소스 코드 3

```
        }

    }
```

서민종 소스 코드 1

```java
package BOJ14585;

import java.util.*;
import java.io.*;

public class Main {
    static int N, M;
    static int[][] map;
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bw = new BufferedWriter(new OutputStreamWriter(System.out));
        StringTokenizer st = new StringTokenizer(br.readLine());
        N = Integer.parseInt(st.nextToken());
        M = Integer.parseInt(st.nextToken());
        map = new int[301][301];
        for(int i=0; i<N; i++) {
            st = new StringTokenizer(br.readLine());
            int x = Integer.parseInt(st.nextToken());
            int y = Integer.parseInt(st.nextToken());
            map[x][y] = M;
        }
        int[][] dp = new int[301][301];
        for(int x=0; x<=300; x++) {
            for(int y=0; y<=300; y++) {
                if(x == 0 && y == 0) {
                    continue;
                }
                int candy = (map[x][y] > x + y) ? M - x - y : 0;
```

서민종 소스 코드 2

```
                if(x == 0) {
                    dp[x][y] = dp[x][y-1] + candy;
                } else if(y == 0) {
                    dp[x][y] = dp[x-1][y] + candy;
                } else {
                    dp[x][y] = Math.max(dp[x-1][y], dp[x][y-1]) + candy;
                }
            }
        }
        int answer = dp[300][300];
        bw.write(answer + "\n");
        bw.flush();
    }
}
```

사건은 다가와 (Easy)(32198)

문제

민정이는 현재 수직선의 원점, 즉 위치 0에 있다. 민정이는 매 시점 수직선 위에서 왼쪽 또는 오른쪽으로 초당 1의 속도로 이동할 수 있고, 정지해 있을 수도 있다.
민정이는 앞으로 사건이 $N(1 \le N \le 100)$번 발생할 것을 알고 있다. 각 사건은 정수 시각 $T(1 \le T \le 1\,000)$에 발생하며, 그때 민정이가 위치 A와 위치 $B(-1\,000 \le A < B \le 1\,000)$에 대해 열린 구간 (A, B) 안에 있으면 카리나의 body bang을 맞게 된다. 위치 A와 위치 B는 안전하다는 점에 유의하라.

각 사건의 T는 모두 다르다.
민정이는 카리나의 body bang을 맞지 않기 위해 적절히 움직이려 한다. 민정이가 body bang을 피할 수 있는지 판별하고, 피할 수 있다면 이동 거리의 최솟값을 구하여라.

예제 입력/출력 1

입력

```
3
10 -3 7
20 -8 2
25 3 9
```

출력

```
8
```

김시온 소스 코드 1

```java
import java.util.*;
import java.io.*;

public class Main {
    static int n;
    static int[][] a;
    static int[][] c;

    static int dp(int t, int x) {
        if (t == 1001) {
            return 0;
        }
        if (c[t][x] != -1) {
            return c[t][x];
        }

        int l = a[t][0];
        int r = a[t][1];
        if (l < x && x < r) {
            // 정답이 될 수 없는 값을 반환
            return c[t][x] = (int) 1e9;
        }

        // 1. 가만히 있기
        int min = dp(t + 1, x);
        // 2. 왼쪽으로 이동
        if (x - 1 >= 0) {
            min = Math.min(min, 1 + dp(t + 1, x - 1));
        }
```

김시온 소스 코드 2

```
        // 3. 오른쪽으로 이동
        if (x + 1 <= 2000) {
            min = Math.min(min, 1 + dp(t + 1, x + 1));
        }

        return c[t][x] = min;
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        n = Integer.parseInt(br.readLine());

        a = new int[1001][2];
        for (int i = 0; i < n; i++) {
            StringTokenizer st = new StringTokenizer(br.readLine());
            int t = Integer.parseInt(st.nextToken());
            int l = Integer.parseInt(st.nextToken());
            int r = Integer.parseInt(st.nextToken());

            a[t][0] = l + 1000;
            a[t][1] = r + 1000;
        }

        c = new int[1001][2001];
        for (int i = 0; i <= 1000; i++) {
            Arrays.fill(c[i], -1);
        }

```

김시온 소스 코드 3

```
        int r = dp(0, 1000);
        if (r == (int) 1e9) {
            System.out.println(-1);
        } else {
            System.out.println(r);
        }
    }
}
```

원찬혁 소스 코드 1

```java
import java.io.*;
import java.util.*;

public class Main {
    static final int inf = 100000000;
    static int n;
    static int dp[][];
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st;
        StringBuilder sb = new StringBuilder();
        n = Integer.parseInt(br.readLine());
        int t = 0, a, b, res = inf, maxt = 0;
        dp = new int[1001][2001];
        for(int i = 0; i <= 1000; i++)
            for(int j = 0; j <= 2000; j++)
                dp[i][j] = -1;
        while(n-- > 0 ) {
            st = new StringTokenizer(br.readLine());
            t = Integer.parseInt(st.nextToken());
            a = Integer.parseInt(st.nextToken()) + 1000;
            b = Integer.parseInt(st.nextToken()) + 1000;
            maxt = Math.max(maxt, t);
            for(int i = a + 1; i < b; i++)
                dp[t][i] = inf;
        }
        for(int i = 0; i <= 2000; i++)
            res = Math.min(res, sol(maxt, i));

```

원찬혁 소스 코드 2

```
30          if(res == inf) System.out.println(-1);
31          else System.out.println(res);
32      }
33
34      static int sol(int t, int loc) {
35          if(loc < 0 || loc > 2000) return inf;
36          if(t == 0) {
37              if(loc == 1000) return 0;
38              return inf; // 불가능
39          }
40          if(dp[t][loc] >= 0) return dp[t][loc];
41          // 정지, 왼쪽, 오른쪽
42          dp[t][loc] = sol(t - 1, loc);
43          dp[t][loc] = Math.min(dp[t][loc], sol(t - 1, loc - 1) + 1);
44          dp[t][loc] = Math.min(dp[t][loc], sol(t - 1, loc + 1) + 1);
45          return dp[t][loc];
46      }
47  }
```

손현준 소스 코드 1

```java
package baekjoon;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.StringTokenizer;

public class P32198 {
    static final int INF = Integer.MAX_VALUE;
    static int[][] dp;
    static boolean[][] visited;
    static HashMap<Integer, Bang> bang;
    static int maxT;

    static class Bang {
        int a, b;

        public Bang(int a, int b) {
            this.a = a;
            this.b = b;
        }
    }

    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st;
```

손현준 소스 코드 2

```java

        int N = Integer.parseInt(br.readLine());
        bang = new HashMap<>();
        maxT = 0;
        for (int n = 0; n < N; n++) {
            st = new StringTokenizer(br.readLine());
            int T = Integer.parseInt(st.nextToken());
            int A = Integer.parseInt(st.nextToken()) + 1000;
            int B = Integer.parseInt(st.nextToken()) + 1000;
            maxT = Math.max(maxT, T);
            bang.put(T, new Bang(A, B));
        }

        dp = new int[maxT + 1][2001]; // dp[시간][위치] > 위치index: 0~999(-1000~-1) - 1000(0) - 1001~2000(1~1000)
        visited = new boolean[maxT + 1][2001];
        // dp 배열 초기화
        for (int i = 0; i <= maxT; i++) {
            for (int j = 0; j <= 2000; j++) {
                dp[i][j] = INF;
            }
        }

        int result = solve(0, 1000);
        System.out.println(result == INF ? -1 : result);
    }

    /*
```

손현준 소스 코드 3

```
     * t: 시간
     * p: 위치
     */
    static int solve(int t, int p) {
        if (t == maxT) return 0;

        if (visited[t][p]) return dp[t][p];
        visited[t][p] = true;

        int result = INF;
        Bang nextB = bang.get(t + 1);
        // 왼쪽 오른쪽 끝을 넘어가지 않도록 처리
        if (p > 0) { // 왼쪽으로 갈 수 있을 때
            int nextP = p - 1;
            if (nextB == null || (nextP <= nextB.a || nextP >= nextB.b)) {
                int temp = solve(t + 1, nextP);
                if (temp != INF) result = Math.min(result, temp + 1);
            }
        }
        if (p < 2000) { // 오른쪽으로 갈 수 있을 때
            int nextP = p + 1;
            if (nextB == null || (nextP <= nextB.a || nextP >= nextB.b)) {
                int temp = solve(t + 1, nextP);
                if (temp != INF) result = Math.min(result, temp + 1);
            }
        }
        // 움직이지 않을 때
        if (nextB == null || (p <= nextB.a || p >= nextB.b)) {
```

손현준 소스 코드 4

```
            int temp = solve(t + 1, p);
            if (temp != INF) result = Math.min(result, temp);
        }

        return dp[t][p] = result;
    }
}
```